8086 Program

WAP to display number entered by user by using array in new line with space.

```
include "emu8086.inc"   ; used to print string
.MODEL SMALL
.STACK 100H
.DATA
   arr db 5 dup(?)
 .CODE
   MAIN PROC
      mov ax, @data
      mov ds, ax
       mov cx,5        ; set counter to 5
      mov si, offset arr  ; address of first number entered is moved into si
       print "Enter 5 number in Array:"
      loopIn:
      mov ah,1    ; mov data in al register
      int 21h
      mov [si],al  ; first number is stored in index of frist address
      inc si
      loop loopIn
       ; To show number in new line
      mov ah,2
      mov dl,10
      int 21h
      mov dl,13
      int 21h ;
       mov si, offset arr           ; address of first number is stored in si
     mov cx,5
```

```
    print "You Enter:"
    loopOut:
     mov dl, [si]
    mov ah,2
    int 21h
     ;To provide space between displayed number
    mov dl ,32
    mov ah, 2
    int 21h     ;
     inc si
    loop loopOut
    mov ax, 4ch
    int 21h
    main endp
end main
```

WAP in assembly language to sort the entered by user in ascending or descending order using an array.

```
include "emu8086.inc"   ; used to print string
.MODEL SMALL
.STACK 100H
.DATA
   arr db 5 dup(?)
 .CODE
   MAIN PROC
    mov ax, @data
    mov ds, ax
    print "Enter 5 number in Array:"
     mov cx,5            ; set counter to 5
    mov bx, offset arr  ; address of first number entered is moved into bx
```

```
mov ah,1            ;input single value
inputs:
int 21h
mov [bx],al         ; first number is stored in frist address
inc bx
loop inputs
 mov cx, 5
dec cx
 outerLoop:
mov bx, cx
mov si,0    ;initialize si
CompLoop:
mov al,arr[si]          ; value stored in first address is placed in al
mov dl, arr[si+1]       ; value stored in second address is placed in dl
cmp al,dl
;jc noSwap       ; To display in ascending order
jnc noSwap       ; To display in descending order
 mov arr[si],dl
mov arr[si+1],al
 noSwap:
inc si
dec bx
jnz CompLoop
 loop OuterLoop
 ; To show number in new line
mov ah,2
mov dl,10
int 21h
```

```
    mov dl,13
    int 21h;
    print "After sorting Array:"
     mov cx, 5
    mov bx, offset arr
    ; This loop to display elements on the screen
    Outputs:
    mov dl, [bx]
    mov ah,2
    int 21h
    ;to provide space between displayed number
    mov dl ,32
    mov ah, 2
    int 21h ;
     inc bx
    loop Outputs
     mov ax, 4ch
    int 21h
    main endp
end main
```

WAP in assembly language to take three numbers from user and display largest or smallest number among three on the screen.

```
.MODEL SMALL
.STACK 100H
.DATA
   msg1 DB 10,13,"Enter First Number:$"
   msg2 db 10,13, "Enter Second Number:$"
   msg3 db 10,13, "Enter Second Number:$"
```

```
msg4 db 10,13, "Largest number is:$"
msg5 db 10,13, "Smallest number is:$"
num1 db ?
num2 db ?
num3 db ?
.CODE
MAIN PROC
   mov ax, @data
   mov ds, ax
    lea dx, msg1
   mov ah, 9
   int 21h
   mov ah,1
   int 21h
   mov num1, al
   lea dx, msg2
   mov ah, 9
   int 21h
   mov ah,1
   int 21h
   mov num2,al
    lea dx, msg3
   mov ah, 9
   int 21h
   mov ah,1
   int 21h
   mov num3,al
   ;lea dx,msg4
```

```
    lea dx,msg5
    mov ah,9
    int 21h
    mov bl, num1
    cmp bl,num2
    ;jng number2    ;jump if number1 is not greater to number2
    jnl number2     ;jump if number1 is not less than number2 to number2
    cmp bl, num3
    ;jng number3
    jnl number3
     mov dl, num1
    jmp display
     number2:
    mov bl,num2
    cmp bl, num3
    ;jng number3     ; jump if number2 is not greater than number3 to number3
    jnl number3     ; jump if number2 is not less than number3 to number3
    mov dl,num3
    jmp display
    number3:
    mov dl, num3
    display:
    mov ah,2
    int 21h
     mov ax, 4ch
    int 21h
    main endp
end main
```

Program to find Largest and Smallest number in an array using assembly language programming.

```
include "emu8086.inc"                    ; To show message in print section
.MODEL SMALL
.STACK 100H
.DATA
   arr DB 8,5,3,6,2
.CODE
   MAIN PROC
      mov ax, @data
      mov ds, ax      ; pass the first address to ds register
      mov si, offset arr   ; the first varible source value is passed into SI
       mov cx, 5       ; create loop that run for five times.
      mov bl, [si]    ; value of SI is passed into bl register
      loopA:
      cmp [si], bl
      ;jle small       ; jump if less or equal to lebel: small
       jge large
       compare:
      inc si
      loop loopA
      ;print "Smallest Value in Array=" ; show the message
       print "Largest Value in Array="
       add bl,48            ; To display the actual decimal number
      mov dl,bl
```

```
    mov ah,2
    int 21h
     ;small:
    large:
    mov bl,[si]
    jmp compare     ; Jump to lebel compare
     mov ax, 4c00h
    int 21h
     main endp
end main
```

WAP in assembly language to read a numbers from user and then display the number whether the number entered by user is even or odd.

```
.MODEL SMALL
.STACK 100H
.DATA
   msg1 DB 10,13,"Enter Number:$"
   msg2 db 10,13, "Number is Even$"
   msg3 db 10,13, "Number is Odd$"
.CODE
   MAIN PROC
      mov ax, @data
      mov ds,ax
      lea dx, msg1
      mov ah, 9
      int  21h
      mov ah,1
      int 21h
       mov bl,2
```

```
    div bl          ; ah contain remainder
    cmp ah, 0
    je even
    lea dx, msg3
    mov ah,9
    int 21h
    mov ah, 4ch
    int 21h
     even:
    lea dx,msg2
    mov ah, 9
    int 21h
    mov ax, 4c00h
    int 21h
    main endp
end main
```

WAP to find whether the number entered by user is positive or negative or zero.

```
.MODEL SMALL
.STACK 100H
.DATA
   msg1 DB 10,13,"Enter Number:$"
   msg2 db 10,13, "Number is Positive$"
   msg3 db 10,13, "Number is Negative$"
   msg4 db 10,13, "Number is Zero$"
.CODE
   MAIN PROC
      mov ax, @data
```

```
mov ds, ax
mov dx, offset msg1
mov ah,9
int 21h
mov ah,1
int 21h
 mov bl,al
mov cl,30h      ; 30 is ASCII code for <or>
cmp bl,cl
jg positive
jl negative
je zero
 positive:
lea dx, msg2
mov ah,9
int 21h
mov ah, 4ch
int 21h
 negative:
lea dx, msg3
mov ah,9
int 21h
mov ah, 4ch
int 21h
 zero:
lea dx,msg4
mov ah,9
int 21h
```

```
        mov ah, 4ch
        int 21h


         main endp
end main
```